Software Requirement Specification (SRS) for Internship Dashboard

1. Introduction

This document, the Software Requirement Specification (SRS), details the complete functional and non-functional requirements for the Internship Dashboard system. The system is a full-stack web application designed to streamline and manage the end-to-end internship workflow between academic **Students** and assigned **Mentors**.

1.1 Purpose of the System

The primary purpose of the Internship Dashboard is to centralize and automate key processes involved in a structured academic internship, including task management, communication, and progress monitoring. It aims to provide a transparent, efficient, and auditable platform for both Students and Mentors to interact and track performance against defined objectives.

1.2 Scope of the Project

The Internship Dashboard system will encompass the following core functionalities:

- **Role-Based Access:** Secure login and distinct dashboards for Students and Mentors.
- **Task Management:** Ability for Mentors to create, assign, and track tasks; ability for Students to view tasks and submit deliverables.
- **Progress Tracking:** Visual representation and tracking of task completion and overall internship status.
- **Data Integrity:** Secure storage and management of user profiles, tasks, and submissions.

The project scope **excludes** features such as integrated video conferencing, automated plagiarism checks, or integration with external payroll/HR systems.

1.3 Definitions and Abbreviations

Abbreviation	Definition
SRS	Software Requirement Specification
MERN	MongoDB, Express.js, React.js, Node.js

Abbreviation	Definition
API	Application Programming Interface
JWT	JSON Web Token
FR	Functional Requirement
NFR	Non-Functional Requirement
UI	User Interface
HTTP	Hypertext Transfer Protocol
OS	Operating System

2. Overall Description

2.1 Product Perspective

The Internship Dashboard is a standalone, full-stack web application. It is not an embedded component of a larger system but is designed to be potentially integrated with an existing Learning Management System (LMS) or academic portal in the future via standard API protocols.

2.2 User Classes and Characteristics

The system supports two primary user classes, each with distinct roles and permissions:

User Class	Description	Key Characteristics	Permissions
Student	The intern enrolled in the program.	Primarily focused on viewing assigned tasks, submitting work, and tracking personal progress.	Task viewing, File submission, Progress monitoring.
Mentor	The academic or industry professional overseeing the intern.	Responsible for task definition, assignment, evaluation, and overall progress supervision.	User management (limited), Task creation/assignment, Submission review, Progress reporting.

2.3 Operating Environment

The application will operate in a standard web environment, utilizing a three-tier architecture (presentation, application, and data).

Component	Technology/Environment	Details
Frontend (Client-side)	React.js	Must be compatible with modern web browsers (Chrome, Firefox, Edge, Safari).
Backend (Application Server)	Node.js with Express.js	Hosted on a cloud platform (e.g., AWS, GCP, Azure).
Database	MongoDB	Non-relational database for flexible data storage.
Operating System	Server: Linux (Ubuntu/CentOS). Client: Any major OS (Windows, macOS, Linux).	Standard OS for deployment and user access.

2.4 Assumptions and Constraints

2.4.1 Assumptions

- All users (Students and Mentors) will have reliable access to the internet and modern web browsers.
- Initial user accounts (Students and Mentors) will be pre-registered by an administrator, or a registration process will be implemented.
- The system will handle file submissions up to a specified size limit (e.g., 50MB per submission).

2.4.2 Constraints

- **Technology Stack:** The system must be developed using the MERN stack (MongoDB, Express, React, Node).
- **Security Standard:** All sensitive data transmission must use HTTPS/SSL/TLS.
- **Compliance:** The system must adhere to all relevant academic data privacy policies.

3. System Features

3.1 User Authentication (SF1)

The system must provide secure mechanisms for user registration and login based on the defined user classes (Student and Mentor).

3.2 Role-Based Dashboard (SF2)

Upon successful authentication, the user must be redirected to a personalized dashboard tailored to their specific role.

- **Student Dashboard:** Focuses on assigned tasks, submission status, and overall progress summary.
- **Mentor Dashboard:** Focuses on a list of assigned Students, task assignment tools, and submission review queue.

3.3 Task Assignment (Mentor → Student) (SF3)

Mentors must have the functionality to create a new task and assign it to one or more specific Students.

3.4 Task Submission (Student → Mentor) (SF4)

Students must be able to view their assigned tasks and submit deliverables (files, links, or text descriptions) against a specific task.

3.5 Progress Tracking (SF5)

The system must track and display the completion status for tasks and overall internship progress.

- **Student View:** Percentage completion, status of individual tasks (Pending, In Progress, Submitted, Graded).
- **Mentor View:** Overview of all assigned Students' progress, highlighting overdue or pending submissions.

4. Functional Requirements

Functional Requirements (FRs) define the specific actions the system must perform.

4.1 User and Authentication Requirements

ID	Description	Role
FR1	The system shall allow a user to register an account using a valid institutional email and password.	All
FR2	The system shall validate user credentials against the database during the login process.	All
FR3	The system shall issue a JWT upon successful login for subsequent API authentication.	All
FR4	The system shall redirect a successfully authenticated user to the appropriate role-based dashboard.	All

4.2 Mentor-Specific Requirements

ID	Description
FR5	The Mentor shall be able to create a new task, including fields for Title, Description, Deadline (Date), and associated files.
FR6	The Mentor shall be able to assign a created task to one or more specific Students.
FR7	The Mentor shall be able to view a list of all submissions for a given task.
FR8	The Mentor shall be able to review a Student's submission and assign a grade or feedback.
FR9	The Mentor shall be able to modify the deadline or description of an unsubmitted task.

4.3 Student-Specific Requirements

ID	Description
FR10	The Student shall be able to view a list of all assigned tasks, including status and deadline.
FR11	The Student shall be able to upload a file or provide a link/text response as part of a task submission.
FR12	The Student shall receive a confirmation upon successful submission of a task.
FR13	The Student shall be able to view the grade and feedback provided by the Mentor for a completed task.
FR14	The Student shall be able to view an overall progress bar indicating the percentage of completed tasks.

5. Non-Functional Requirements

Non-Functional Requirements (NFRs) specify criteria that can be used to judge the operation of a system, rather than specific behaviors.

5.1 Performance (NFR1)

- **Response Time:** The system shall display the dashboard view within 3 seconds under normal load (up to 100 concurrent users).
- **Query Time:** All database queries for task retrieval and status updates shall be executed within 500 milliseconds.

5.2 Security (NFR2)

- **Authentication:** All user authentication shall utilize industry-standard JSON Web Tokens (JWT) transmitted over HTTPS.
- **Data Protection:** All user passwords shall be stored as one-way hashed values (e.g., using bcrypt).
- **Authorization:** The system shall enforce role-based access control (RBAC), preventing Students from accessing Mentor-only APIs and vice-versa.

5.3 Scalability (NFR3)

- The system architecture (MERN stack) shall support vertical and horizontal scaling (e.g., load balancing Node.js instances) to accommodate a growth of up to 5,000 active users (Students + Mentors) without major re-architecture.
- The MongoDB schema shall be designed to facilitate high-volume read operations for dashboard summaries.

5.4 Usability (NFR4)

- The User Interface (UI) shall be intuitive and require minimal training for both user classes.
- The UI shall be fully responsive, providing optimal display on desktop and tablet devices.

6. External Interface Requirements

6.1 User Interface Description

The UI will be a Single Page Application (SPA) developed with React.js. It will feature a modern, clean design with distinct layouts for Student and Mentor dashboards. Key UI components include:

- **Navigation Bar:** Role-specific links (Tasks, Submissions, Profile, Logout).
- **Student Dashboard:** Task list with sort/filter options, a prominent progress visualization, and notifications.
- **Mentor Dashboard:** Student list, "Assign New Task" button, and submission review queue.

6.2 API Interactions

The backend (Node.js/Express) will expose RESTful APIs consumed by the frontend. All API interactions will be stateless and authenticated via JWT.

API Example	Method	Description	Authentication
/api/tasks	GET	Retrieve all tasks for the logged-in user.	JWT Required
/api/tasks/assign	POST	Assign a new task to a Student (Mentor only).	Mentor Role
/api/submissions	POST	Submit work for a specific task (Student only).	Student Role

6.3 Database Interactions

The application will use a NoSQL database (MongoDB). The backend will handle all database interactions, ensuring data validation and integrity before storage or retrieval. Direct client-side access to the database is strictly prohibited for security reasons.

7. System Architecture

7.1 MERN Stack Explanation

The Internship Dashboard will be built on the MERN stack, ensuring a consistent technology environment across the entire application lifecycle.

Layer	Technology	Role in System
Frontend (Presentation)	**React.js**	Handles the user interface, state management, and interaction logic. Communicates with the backend via REST APIs.
Backend (Application)	**Node.js + Express.js**	Manages routing, API endpoints, business logic, authentication, and communication with the database.
Database (Data)	**MongoDB**	Stores all application data (Users, Tasks, Submissions) in JSON-like documents.

[A diagram illustrating the flow of data between the React frontend, Node.js/Express backend, and MongoDB database, emphasizing the API layer]

7.2 Data Flow Between Student and Mentor

1. **Task Assignment:** Mentor creates Task → Express API validates → MongoDB stores Task → Student dashboard reads new Task.
2. **Task Submission:** Student uploads File → Express API processes and saves File location/metadata → MongoDB updates Task status to 'Submitted'.
3. **Review/Grading:** Mentor retrieves Submission → Mentor provides Feedback/Grade → Express API updates Submission document → Student dashboard displays updated status.

8. Use Case Descriptions

8.1 Use Case: Login

Attribute	Value
Use Case ID	UC-L01
Actor	Student, Mentor
Goal	Securely authenticate the user and grant access to the system.
Preconditions	User must have an existing account.
Success Scenario	1. User enters credentials. 2. System validates credentials. 3. System issues a JWT. 4. User is redirected to the role-based dashboard.

8.2 Use Case: Assign Task (Mentor)

Attribute	Value
Use Case ID	UC-M01
Actor	Mentor
Goal	Create a new task and assign it to one or more Students.
Preconditions	Mentor must be logged in.
Success Scenario	1. Mentor navigates to the 'Assign Task' page. 2. Mentor enters task details (FR5). 3. Mentor selects target Students (FR6). 4. System saves the task and notifies selected Students.

8.3 Use Case: Submit Task (Student)

Attribute	Value
Use Case ID	UC-S01

Attribute	Value
Actor	Student
Goal	Submit required deliverables for an assigned task.
Preconditions	Student must be logged in and the task deadline must not be passed.
Success Scenario	1. Student views the task details (FR10). 2. Student uploads file(s) or enters response (FR11). 3. Student clicks 'Submit'. 4. System validates and saves the submission (FR12).

8.4 Use Case: View Dashboard

Attribute	Value
Use Case ID	UC-V01
Actor	Student, Mentor
Goal	View personalized, real-time overview of current internship status.
Preconditions	User must be logged in.
Success Scenario	1. User accesses the dashboard (SF2). 2. System retrieves relevant data (tasks for Student, submissions for Mentor). 3. System displays progress metrics and task lists (SF5).

9. Database Design

The system will utilize a MongoDB database, employing a collection-based structure for efficient data management.

9.1 Collections and Schema Explanation

Collection	Description	Key Fields
Users	Stores all Student and Mentor profiles and authentication data.	`_id`, `email (unique)`, `password (hashed)`, `role (enum: 'Student', 'Mentor')`, `name`
Tasks	Stores details of all created tasks.	`_id`, `title`, `description`, `deadline`, `mentor_id (ref: Users)`, `assigned_to ([ref: Users])`
Submissions	Stores the link between a Student, a Task, and the submission details.	`_id`, `task_id (ref: Tasks)`, `student_id (ref: Users)`, `submission_data (file link/text)`, `status (enum)`, `grade`, `feedback`

10. Constraints and Limitations

- **File Size Limit:** The maximum size for any single uploaded file submission is restricted to 50MB.
- **Time Zone:** All deadlines and timestamp displays will be handled consistently in UTC on the server and converted to the user's local time zone on the client.
- **Browser Support:** The system guarantees full functionality only for the latest two stable versions of Chrome, Firefox, and Edge.

11. Future Enhancements

The following features are considered outside the current scope but are planned for future versions:

- **Integrated Communication:** A direct messaging feature between Students and their assigned Mentors.
- **Notifications System:** Real-time email or in-app notifications for new tasks, submission deadlines, and grading.
- **Reporting:** Automated generation of progress reports in PDF format for administrators.
- **Calendar Integration:** Integration with external calendar applications (Google Calendar, Outlook) for task deadlines (Calendar event).